

## 07. Dynamic Imports

### Problem

Normal (static) `import/export` me:

- Path sirf ek fixed **string** ho sakta hai (function call nahi).
- `if` ya kisi block ke andar import **nahi** kar sakte.

Ye isliye ki tools code structure analyze karke bundling aur tree-shaking kar sakein.

### Solution: `import()` expression

Kisi bhi jagah se, kabhi bhi module load karo. Ye **promise** return karta hai jo module object (saare exports) ke saath resolve hota hai.

```
js

let modulePath = prompt("Which module?");

import(modulePath)
  .then(obj => { /* module object */ })
  .catch(err => { /* loading error */ });
```

### `await` ke saath

```
js

// say.js
export function hi() { alert('Hello'); }
export function bye() { alert('Bye'); }

// main
let { hi, bye } = await import('./say.js');
hi(); bye();
```

### Default export ho to

`default` property se milta hai:

```
js

let obj = await import('./say.js');
obj.default();
// ya: let { default: say } = await import('./say.js');
```

## Yaad rakho

- Regular script me bhi chalta hai, `type="module"` zaruri nahi.
- `import()` function jaisa dikhta hai par **function nahi hai** (special syntax hai, `super()` ki tarah). Isliye variable me copy ya `call/apply` nahi kar sakte.
- Use case: jab module sirf zarurat par (on-demand / conditionally) load karna ho.